Food Store en la feria

TP8 · Unidad 8 — La base de la tablet

Equipo: Avalos Pablo · Blangetti Sofia · Suarez Ismael
Fecha: 7 de noviembre de 2026

Informe revisado y aprobado por el equipo

Implementamos una base SQLite para consultar el catálogo sin conexión, guardar borradores y simular su evaluación contra una instantánea de PostgreSQL. El trabajo integra el modelo relacional de U1–U6 y la representación documental de U7, sin construir una aplicación ni una sincronización comercial real.

> **El móvil propone, el servidor dispone.** PostgreSQL es la fuente de verdad de precios, stock, usuarios y ventas. `SIMULADO_APROBADO` no es una venta ni una reserva: no crea pedidos ni descuenta stock.

1.a · Qué viaja a la tablet

Origen	Dato recibido / uso local
`producto.id, nombre`	`producto_id, nombre`: identificar y mostrar el catálogo.
`producto.precio`	`(precio * 100)::BIGINT` → `precio_centavos`; entero, sin aritmética monetaria flotante al guardar.
`stock, disponible`	`stock_visto, disponible_visto`; información observada, no autorización para vender.
Baja de producto o categoría	`producto.eliminado OR categoria.eliminado` → `eliminado`; conserva la fila como lápida.
`now()` de la exportación	`capturado_en`: fecha de la copia, no hora histórica de evaluación de cada solicitud.

No viajan: descripción, imagen, `categoria_id` ni `created_at` de producto; la tabla categoría (salvo su baja combinada), perfiles de usuario, mails, celulares, contraseñas, pedidos, detalles ni objetos programables. Solo se lleva lo necesario para consultar y proponer, evitando copiar datos personales.

Los UUID, cantidades, forma de pago, precios vistos y estados nacen localmente. `usuario_id_servidor` es una referencia numérica: no copia el perfil ni tiene FK local a usuarios; su validez le correspondería al servidor. En D se usó el id 1 leído del seed de pruebas.

Fuentes: U8/docs/integracion_motores.md §§1–3; U8/postgresql/snapshot_movil.sql:6–15; U8/movil/schema_sqlite.sql:21–63. «U8» significa unidades/unidad-8/ desde la raíz del repositorio.

Esquema, transacciones e instantánea

Objeto SQLite	Responsabilidad y reglas
`producto_cache`	PK de producto, centavos y stock no negativos; banderas 0/1; fecha de captura.
`pedido_borrador`	UUID local, referencia numérica de usuario, fecha y pago limitado a EFECTIVO, TARJETA o TRANSFERENCIA.
`item_borrador`	FK a borrador y catálogo; PK compuesta; cantidad > 0 y precio visto no negativo.
`cola_sync`	PK `solicitud_uuid`, FK al borrador; cuatro estados; índice parcial: una sola PENDIENTE por borrador.

Las cuatro tablas son `STRICT` y cada conexión activa `PRAGMA foreign_keys = ON`. `snapshot_servidor` es la quinta tabla, agregada en C. El borrador completo se guarda con `BEGIN … COMMIT`; los fallos no deben dejar encabezados o ítems parciales.

Pruebas B históricas: COMMIT, ROLLBACK, C1 sin conexión y C7 con errores de CHECK/FK terminaron según lo esperado. Las **33 comprobaciones SQL** no son 33 casos de la consigna [B:25–93]. No se repitieron en esta redacción.

Flujo: PostgreSQL → vista `v_snapshot_movil` → CSV y metadatos → `snapshot_servidor` → UPSERT de `producto_cache`. La exportación usa una transacción de lectura; la renovación agrupa DELETE, importación, validaciones y UPSERT en una sola transacción. Convierte `t/f` a `1/0` y conserva precios vistos e historial de cola. El CSV admitido tiene siete campos simples, sin entrecomillado ni comas o saltos internos.

Semáforo: solo PENDIENTE. Rojo: ausencia, baja, no disponibilidad o stock insuficiente; sin rojo, amarillo por precio distinto; si todo coincide, verde. Guarda estado y motivo solo en SQLite. Las filas ausentes de la instantánea y las lápidas se distinguen en la página 5.

Cómo leer los casos y sus referencias

Comandos de `ejecutar_bloque_d.sh` y `bloque_d.sh`, con estas variables. **No se ejecutaron al redactar; no repetir sobre la base entregada.**

```
U8=unidades/unidad-8
D="$U8/movil/bloque_d.sh"
DB="$U8/movil/foodstore_movil.db"
RUN=20261107_195358
```

Las referencias [código:líneas] remiten a estos archivos, relativos a U8/evidencias/. Son registros históricos; no una nueva ejecución ni una revisión humana.

```
B  = bloque_b_20261107_121805.txt
C  = bloque_c_sqlite_20261107_130343.txt
D  = bloque_d_entregable_20261107_195358.txt (detenido)
DC = bloque_d_entregable_continuacion_c6_20261107_195500.txt
T  = bloque_d_controles_copias_20261107_195510.txt
E  = bloque_e_lectura_base_final_20261107_203411.txt
```

C2 y C3: preparar no es simular

Los cambios de escenario se hicieron en PostgreSQL de **pruebas**, reconstruido desde U6 más datos auxiliares sintéticos. Los ids 4–7 fueron devueltos al instalar esos datos; no se suponen ids universales. Cada simulación lee el control del servidor antes y después de evaluar en SQLite.

C2 · Precio cambiado y aceptación posterior

```
bash "$D" borrador "$DB" c2
bash "$D" preparar c2
bash "$D" instantanea c2 "$RUN" "$DB"
bash "$D" simular "$DB"
bash "$D" aceptar "$DB" c2
bash "$D" simular "$DB"
```

Preparación: primero se creó el borrador (producto 4, una unidad a 120000 centavos); luego `preparar_c2_precio.sql` subió 1200,00 → 1320,00 en el servidor de pruebas. `instantanea` exportó `c2_precio` y renovó el catálogo; el ítem conservó 120000 [D:60–169].

Resultado: la primera solicitud quedó en `CONFLICTO`, con motivo «precio_visto=120000 centavos, precio_actual=132000 centavos» [D:171–246]. El precio que vio el cliente no se sustituye silenciosamente al renovar el catálogo.

Aceptación simulada: `aceptar_precio.sql` actualizó el ítem a 132000 y creó una solicitud con UUID nuevo dentro de la misma transacción. La nueva quedó `SIMULADO_APROBADO`; la original siguió en CONFLICTO con su motivo histórico [D:252–393]. No se registró una venta.

C3 · Stock insuficiente

```
bash "$D" borrador "$DB" c3
bash "$D" preparar c3
bash "$D" instantanea c3 "$RUN" "$DB"
bash "$D" simular "$DB"
```

Preparación: el borrador pidió tres unidades del producto 5 a 90000 centavos. `preparar_c3_stock.sql` redujo el stock del servidor de 10 a 2 y se exportó/renovó `c3_stock` [D:395–479].

Resultado: `RECHAZADO`, motivo «producto_id=5: stock disponible=2, cantidad solicitada=3» [D:481–556]. El stock visto no reserva unidades: la cantidad propuesta supera la instantánea actual aunque el producto siga visible.

Controles que protegen la explicación

En copias descartables se comprobó que un rechazo por stock tiene prioridad sobre otro ítem con cambio de precio; también se rechazó una segunda PENDIENTE por borrador. La aceptación con producto ausente se detuvo antes del UPDATE y un INSERT fallido revirtió el UPDATE previo [T:120,142,171,191–195]. Son pruebas sobre copias, no resultados adicionales de la base final.

Trazabilidad de comandos: U8/movil/ejecutar_bloque_d.sh, funciones casos_c2_c5; U8/movil/bloque_d.sh, operaciones borrador/preparar/instantanea/simular/aceptar. La instalación e ids constan en bloque_c_postgresql_20261107_125121.txt:125–148.

C4, C5, C6 e invariancia

C4 · Baja lógica con lápida

```
bash "$D" borrador "$DB" c4
bash "$D" preparar c4
bash "$D" instantanea c4 "$RUN" "$DB"
bash "$D" simular "$DB"
```

Se creó el borrador antes de la baja del producto 6. `preparar_c4_baja.sql` cambió `eliminado` a true y se renovó `c4_baja`. Resultado: `RECHAZADO`, «producto_id=6: eliminado=t». La fila quedó con `eliminado=1`, fuera del catálogo visible; el precio visto siguió en 70000 [D:558–740].

C5 · Sin discrepancias

```
bash "$D" borrador "$DB" c5
bash "$D" simular "$DB"
```

Dos unidades del producto 7 a 50000 centavos dieron `SIMULADO_APROBADO` [D:742–838]. No hubo mutación ni nueva exportación propia de C5: se usó la instantánea vigente `c4_baja`, sin discrepancias para ese producto. No es venta ni reserva de stock.

C6 · Reenvío del UUID y repetición del semáforo

```
bash "$D" reinsertar "$DB" c6_uuid
bash "$D" simular "$DB"
```

C6-1: reinsertar el UUID original de C2 produjo `UNIQUE constraint failed: cola_sync.solicitud_uuid`; SQLite devolvió 1 (esperado), el auxiliar validó el error y las cinco filas permanecieron iguales [D:843–862; DC:118–120]. **C6-2:** repetir el semáforo procesó 0 y no alteró el estado consultado ni el CONFLICTO original [DC:147–209]. En producción la protección debería estar también en el servidor: registrar el UUID recibido de forma única y transaccional para impedir un doble procesamiento; eso no está implementado.

Consulta de control en PostgreSQL

```
SELECT 'producto', id, precio, stock, disponible, eliminado
FROM producto ORDER BY id;
SELECT 'pedidos', count(*), coalesce(sum(total), 0) FROM pedido;
SELECT 'detalle_pedido', count(*), coalesce(sum(cantidad), 0) FROM detalle_pedido;
```

Son las consultas de `postgresql/control_invariancia.sql`, leídas en REPEATABLE READ READ ONLY por `simular`. Las salidas antes/después coincidieron en las seis simulaciones: 9 productos controlados, 5 pedidos (total 14700,00), 7 líneas (14 unidades) [D:171–225,318–372,481–535,665–719,763–817; DC:128–179]. **Se compararon esas salidas, no PostgreSQL completo byte a byte.** La preparación sí cambió precio, stock y baja deliberadamente antes de cada control.

Incidencia conservada: D se detuvo en el paso 32 por ubicar mal archivos de control [D:867–872]. La continuación comparó las capturas de C6-1 y ejecutó C6-2 [DC:3–9,118–254]. No existe una ejecución íntegra desde cero de la secuencia corregida.

Tres motores, una autoridad comercial

Motor	Qué guarda	Quién manda / límite
PostgreSQL U1–U6 / copia U8	`categoria`, `usuario`, `producto`, `pedido`, `detalle_pedido` y objetos programables. U8 agrega productos de prueba y la vista de instantánea.	Fuente de verdad de catálogo, precios, stock, usuarios y ventas. El laboratorio U8 no es producción.
MongoDB · U7	Colección `pedidos`, subconjunto documental con líneas embebidas y precio histórico de `detalle_pedido.precio_unitario`.	Representación de consulta cargada desde PostgreSQL. No lo reemplaza ni se sincroniza permanentemente; no participa en U8.
SQLite · U8	Catálogo e instantánea; borradores, ítems y cola locales.	Solo sus propuestas e historial local. No confirma ventas, precios ni stock del servidor.

Fuente: U8/docs/integracion_motores.md §§1–3. Referencias fijas: U6 `u6-cierre = 8f7877c8a1ff25c1be28b29c46c38cd7c9663e86` en ../foodstore_baseII; U7 `u7-cierre = 703583e25ca9ca6ad0ab6be147d4c3b42f41c4e4` en este repositorio.

Qué respaldar y cómo comprobar la recuperación

PostgreSQL: procedimiento U6, dump completo `pg_dump -Fc` con línea base; `restore_test.sh` restaura en una base nueva y compara controles. En U8 se propone agregar la validación de existencia de `v_snapshot_movil` y una fila por producto. El ensayo histórico de H1 validó datos en modo `sin-propietarios`; no acreditó recuperación de propietarios, ACL, roles ni seguridad de U5. No hubo restauración nueva en E.

MongoDB: reconstruible desde `unidades/unidad-7/datos/pedidos_origen.json` y scripts U7; no se ensayó aquí. **SQLite:** propuesta de copia completa cerrada al cierre de turno y antes de renovar, con SHA-256 y control de integridad. Probar una copia aparte: huella coincidente, `integrity_check=ok`, `foreign_key_check` sin filas y estado igual al registrado. La instantánea no recupera borradores ni cola perdidos.

Si PostgreSQL vuelve a un respaldo antiguo

Propuesta: descargar y renovar la instantánea antes de evaluar PENDIENTE. El UPSERT deja sin cambios filas del cache ausentes de la instantánea: pueden seguir visibles, aunque el semáforo rechace un ítem ausente. No modifica precios vistos ni reevalúa solicitudes resueltas. Revisar manualmente las potencialmente afectadas y conservar su historial; cualquier solicitud nueva requiere decisión explícita y UUID nuevo. `creada_en` no es fecha de evaluación, por lo que no se promete una selección automática histórica precisa.

Quién revisaría la cola y cuándo (propuesta, no acordada)

CONFLICTO: vendedor con el cliente, en el momento o al cierre del turno. **RECHAZADO:** responsable comercial, diariamente al cierre y después de restaurar PostgreSQL. Conteos por tablet: revisión semanal del responsable comercial. La consulta de revisión no cambia estados evaluados.

Fuente: U8/docs/mantenimiento_u8.md §§1–5. El resguardo local y las nueve capturas C6 conservadas **no son un ensayo de restauración**; el complemento registra 48 huellas coincidentes (rc=0), no una recuperación [U8/evidencias/bloque_e_complemento_20261107_212101.txt:170–200].

Revisión asistida por IA

Inventario: **115 lecturas completas + 5 correspondencias históricas U7 = 120 scripts**, sin lecturas pendientes; no son 120 ejecuciones nuevas. Incluye una lectura comunicada de ChatGPT/Mila con blob cotejado. H1/H3/H6: selección aprobada por el equipo para el informe. La consigna pide OpenCode Plan; el equipo informó sustitución por Claude, no aprobación docente del informe.

Hallazgo / decisión	Comprobación, motivo y límites
H1 · U6 «Restaurar requiere el origen». **Descartado** en el caso ensayado.	El 05/11/2026 se renombró el origen: `pg_database` mostró «u6_fix_origen|ausente»; restauración `RESTAURACION_VALIDADA`, exit=0 [R1:122–130,158–172]. `restore_test.sh` ensayado: SHA `68031b1f…`, commit `451718f`; cierre: `973223df…`. El diff `451718f..u6-cierre` solo cambia cuatro líneas de comentarios/mensajes, no conexión ni restauración. Límite: caso y modo sin-propietarios ensayados; el cierre se sostiene por el diff, no por otra ejecución.
H3 · U3 Índice por `lower(mail)`. **Aceptado** como costo observado, no defecto.	`idx_usuario_mail_lower`, PostgreSQL 17.11, 20/09/2026; `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)`: tres corridas, primera de calentamiento, media de 2 y 3. INSERT de 1000 usuarios con BEGIN/ROLLBACK: **7,6655 → 14,7085 ms; +91,88 % observado**. Lectura: 10,8865 → 0,136 ms [R3:97–104,207–208,258,288–302]. Script/evidencia: `da5f3e4`. Se recrearon tres candidatos; solo uno sobre usuario. Dos corridas válidas, cachés y otros efectos no aíslan un costo universal. No se propone eliminarlo: depende de la carga.
H6 · U8 «Reevalúa resueltas». **Descartado** para el filtro y C6-2 documentados.	`semaforo.sql`, SHA `2635767318a7…`, filtra PENDIENTE al calcular/actualizar (56,103–109) y controla evaluadas (125–129). C6-2: 0 procesadas, estado consultado idéntico y CONFLICTO original conservado [DC:147–154,177–209]. No es volcado íntegro: omite `pedido_borrador.creado_en` y resume la instantánea. No prueba idempotencia del servidor ni otros casos/versiones.

Conclusión y pendientes

La base entregable es la **posterior a D**: 4 borradores y 5 solicitudes (2 SIMULADO_APROBADO, 1 CONFLICTO, 2 RECHAZADO, 0 PENDIENTE), con integridad y correspondencia de siete campos con `c4_baja` comprobadas en la lectura histórica [E:41–48]. Su SHA-256 es:

```
cf541fa5974d16f884c0d897916b7c72964b5171d7f89329f531d818a3a12ace
```

E sigue abierto: revisión humana individual de los scripts, DUIA contra formato oficial, elección y ensayo de defensa de 10 minutos, modalidad ZIP o repositorio con etiqueta tp8 y operaciones Git. Ismael comunicó la aprobación del equipo al informe. No se declara aprobación docente, DUIA definitiva, ensayo realizado ni entrega publicada.

Referencias de 3.c: R1 = ../foodstore_basell/unidades/unidad-6/evidencias/correccion_restauracion_20261105.txt (huella ensayada:17); R3 = ../foodstore_basell/unidades/unidad-3/informes/evidencia_modelo_oficial.md. Desarrollo y límites: U8/docs/revision_scripts_ia.md §§3–4; totales: U8/docs/revision_inventario_scripts.md, «Totales». Las pruebas U3/U6 son históricas, no ejecuciones nuevas de E. H5 no se usa como prueba del .import original: C:219–227 solo acredita el rechazo previo del auxiliar, sin abrir la base para renovar. Marco: consigna oficial TP8 (Unidad 8, ciclo 2026), §§1–4 y «Qué se entrega»; especificación: U8/docs/kiro/; uso de IA: U8/docs/bitacora_ia.md y DUIA.md (borrador).